Contents

Restaurant OS — Full System Audit

Date: July 27, 2026 **Audited:** github.com/nicholaswittle/restOS @ commit `ffd8f5e` — archive snapshot of the `apex_v2` Flutter app, `jigsys_site`, and the archived `jigsy` ordering app **Method:** Full read of all 4 design documents, all 6 Supabase migrations, all 3 edge functions, and the new OS-tier feature code (ordering, call-out engine, smart capacity, labor-vs-revenue), plus live competitor research. Companion to the earlier apex_v2 code audit.

---

1. The Verdict, Up Front

Yes — this is well designed, and yes, it can actually work. The architecture is sound, the strategy is unusually disciplined, and the competitive analysis in the design docs checks out against real 2026 pricing and review data. The "nobody else sells this" claim is substantially true for the independent-restaurant segment.

But there is one gap between the dream and the build that matters more than everything else combined: **the flagship feature — "one number tells you if the night made money" — currently only sees online-order revenue, which is a small fraction of a restaurant's actual sales.** Until POS integration or a daily revenue entry exists, the labor-vs-revenue dashboard will show a misleading number. That is fixable, and the plan already knows it (POS integration is deliberately deferred), but it is the difference between a demo and a product an owner trusts.

Four smaller defects also need fixing before a paying pilot (Section 5).

---

2. How It All Works Together, in Plain Language

Forget the code for a minute. Here is the whole system as a story.

One database, one app, many faces

Everything — the staff schedule, the menu, the orders, the clock punches, the tips, the log book — lives in **one Supabase (Postgres) database**. Every row is tagged with an `organization_id` (the restaurant), and row-level security (RLS) policies make sure one restaurant can never see

another's data. This was the single best architectural decision in the project: instead of "build now, connect later," the OS emerges for free because all the data was always in one place.

On top of that database sits **one Flutter app** (iOS, Android, web). The app has a module registry — each feature (Schedule, Tips, Log Book, Labor Cost, Ordering, Chat, etc.) is a plug-in. What you see when you open the app depends on two things: **what tier the venue bought** (Free / Pro $25 / OS $99 / Multi $199) and **what role you hold** (owner, manager, server, kitchen, readonly). Entitlements are per-module, so a venue can also buy just one piece ("plug in the log book, skip the rest").

The three pillars

Pillar 1 — The staff side (what Apex already is). Employees open the app to a personal dashboard: your next shift, your hours this week, your estimated pay and tips, shift notes from the last crew, one-tap clock-in. Managers build schedules (including by photographing a paper schedule and letting AI parse it), get labor-law guardrails (minor hours, overtime, break warnings), and handle swaps and time-off. Clock-in uses a QR code on the wall plus an offline punch queue for bad kitchen Wi-Fi.

Pillar 2 — The customer side (ordering). A restaurant gets a public menu link (no login needed — an unguessable `public_token` opens it). Customers browse the menu with modifiers (crust, sauce), build a cart, and place pickup orders. The staff console lets the kitchen accept/reject orders, pause ordering, mark items sold out. Orders are realtime — they appear the moment they're placed.

Pillar 3 — The OS bridge (the part nobody else has). Because pillars 1 and 2 share one database, three features emerge that competitors structurally cannot copy:

- **Labor vs. revenue:** clock punches × hourly rates vs. order totals, same night, same screen. "$2,400 sales, $480 labor = 20%."
- **Smart capacity:** ordering checks who's actually clocked in before accepting orders. Understaffed ⟶ the system warns customers about longer waits, or auto-pauses ordering entirely. Staffed back up ⟶ it resumes.
- **No-show call-out engine:** a staff member taps "can't make it" ⟶ an edge function finds eligible coworkers (right role, not on approved time-off, not already working that day) ⟶ texts and pushes them ⟶ first to claim gets the shift ⟶ if nobody fills it, ordering capacity adjusts down.

Supporting all three: a **notification router** (push first, SMS fallback via Twilio, quiet hours, per-user preferences) that exists specifically because unreliable notifications are the #1 complaint about every competitor.

What's in the repo vs. what's deferred

Status	Items
Built	All of Pillar 1 (16 scheduling features), ordering platform with menu/cart/console, labor-vs-revenue dashboard, call-out engine + SMS fan-out, capacity engine, tip pools, log book, chat, notification routing, 3 edge functions, 6 migrations, demo mode, 28 passing tests
Designed, deliberately deferred	POS integration (6 tables incl. idempotency keys + dead-letter queue), ML prep-time prediction (data capture only), floor plan/table management, multi-location
Not yet designed	Online payment (orders are `payment_mode: 'manual'` — pay at pickup), payroll export beyond CSV

---

3. Is It Well Designed?

Genuinely yes, and the reasons are specific:

1. **The build order is the design.** Every phase ships something useful standalone (log book, tips, labor cost) before any OS connection exists. The docs explicitly analyze why past projects died ("built platform before any feature") and organize this one to avoid it. This is the rare plan where the strategy section is as good as the code.
2. **One backend, org-scoped from day one.** The "New Rule" in the build-order doc — every restaurant feature lives in the same Supabase with `organization_id` — is what makes the OS features *emergent* rather than integration projects. Smart capacity is ~230 lines of code because the data was already co-located.
3. **RLS is treated as a first-class design surface.** Helper functions (`is_member`, `has_role`), idempotent migrations with rationale in comments, a partial unique index guaranteeing one open call-out per shift, a unique index preventing double tip payouts, and a clever confirmation-token pattern so anonymous customers never need a SELECT policy on orders.
4. **Money math is done carefully.** Tip splits use largest-remainder allocation so cents always reconcile to the pool total. Clock punches are full timestamps, never clock-face strings. Cents are integers everywhere.
5. **The multi-vertical thinking is correctly restrained.** The master plan keeps the door open to salons/trades/retail ("a vertical is a pack, not a fork") but explicitly refuses to build vertical two until vertical one pays. That is exactly the right amount of generality.

6. **Demo mode at the HTTP layer** means sales demos run the real screens against seeded data with zero demo branches in feature code.

---

4. Would It Actually Work? The Honest Gaps

Ordered by how much they matter.

4.1 The revenue denominator problem (the big one)

The labor-vs-revenue dashboard computes `revenue` **only from `online_orders`**. For a brewpub, online pickup might be 10–20% of a night's sales; dine-in and bar sales flow through the POS and never touch this database. So the flagship number — labor as % of revenue — will read something like 120% instead of 20%. An owner who sees that once stops trusting the dashboard forever.

Fixes, cheapest first: (a) add a daily revenue entry field ("what did the POS say tonight?") — one column, one tap, immediately makes the number real; (b) later, the deferred Phase 4 POS integration (Square is the natural first target — free API, common in independents; note Toast charges $50/mo for API access, charged by Toast, per 2026 pricing trackers). The design docs already defer POS integration correctly; the miss is not shipping the manual-entry stopgap with the dashboard itself.

4.2 Public checkout is likely broken as written (verify before pilot)

Anonymous customers insert orders directly. But the `order_items` insert policy checks `exists (select 1 from online_orders ...)` — and Postgres applies the `online_orders` RLS *inside* that subquery. Anonymous users have no SELECT policy on `online_orders`, so the EXISTS always evaluates false for them, and every order_items insert is rejected. In plain terms: **a customer can create an empty order but cannot add any food to it.** If the demo ever ran against a real backend this would have failed loudly, so either it was only exercised in demo mode or there's an untested path. Fix with a `place_order` RPC (security definer) that inserts order + items + modifiers atomically — which also fixes the next issue.

4.3 Order totals are computed by the customer

The cart sends its own `subtotal_cents` / `total_cents` and the server trusts them. Since payment is manual (pay at pickup), a tampered client can order a $24 tray for $0.01, or order-bomb the kitchen with fake tickets — the anon insert policy only checks "not paused." Any public, unauthenticated write endpoint will get abused eventually. The `place_order` RPC should recompute prices from the menu tables server-side, enforce pickup time windows, and rate-limit by device/IP.

4.4 The "killer feature" isn't wired up

`CapacityEngine.autoAdjust()` — the code that actually auto-pauses ordering when staffing drops — **is never called from anywhere.** The capacity screen calls `check()` only. And even once wired, the engine counts *all* clocked-in staff (servers, hosts) against capacity, while the design doc's rule is "orders per **cook**." A night with 4 servers and 0 cooks would read as full capacity. Two fixes: call autoAdjust from a real trigger (a scheduled edge function or a database webhook on `time_entries`, not a screen someone has to open), and count only kitchen roles.

4.5 Carried over from the apex_v2 audit

Still open in this snapshot: staff keyed by display name (two "Sams" collide), clock-QR forgeable by photo all day, tip publish can wedge mid-transaction, offline queue can jam permanently on one bad row, and the `messages`/`has_role` RLS issues. None are OS-tier, but all touch money or identity — fix before payroll and payouts get real.

4.6 Smaller notes

- **Payments are manual-only.** Fine for the pilot (pay at pickup), but "manual" should be a labeled v1 limitation in the pitch, not a surprise.
- **SMS costs scale with the call-out engine.** Fan-out texts every eligible coworker; Twilio is ~$0.008/message — trivial at one venue, worth a per-org cap and quiet-hours respect at fifty.
- **Edge functions are now in the repo** (the earlier audit flagged their absence) — good. Keep them versioned with the migrations.
- **`restaurants` public SELECT exposes every venue's row** (names, tokens) to anyone with the anon key. Low harm, but scope it to a lookup-by-token RPC.

---

5. The Competition, Checked Against Reality

I verified the design docs' competitive claims against current (2026) pricing trackers and user reviews. The claims hold up well.

Pricing reality (2026, verified)

Product	Real cost for a single independent venue	What you actually get
7shifts	Free tier (≤1 location); Essentials ~$40–45/loc/mo; Pro ~$80–90; Premium ~$135–150	Scheduling + time clock. **Tip management is a $49.99/mo add-on; Manager Log Book $14.99/mo add-on** on lower tiers (included only at Premium)
Homebase	Free (≤10 employees); Essentials $24/loc/mo; Plus $56; payroll +$39 + $6/employee	General SMB tool — no restaurant-specific tip pooling or POS-driven labor forecasting
Deputy	~$5–9 **per user**/mo ⟶ $100–180/mo for 20 staff	Multi-industry; compliance engine; per-user pricing punishes restaurant headcount
When I Work	~$5 **per user**/mo	Generic; no restaurant features
Toast	~$1,000+/mo all-in with hardware lock-in; API access alone costs $50/mo	The full-stack enterprise option — POS + scheduling + ordering, at enterprise prices
Square	POS + Square Online + Square Appointments are *separate products that don't talk to each other*	Pieces exist, connection doesn't
restOS Pro / OS	**$25 / $99 flat**	Scheduling + time clock + tips + log book + labor cost (Pro); + ordering + smart capacity + call-out engine + labor-vs-revenue (OS)

Sources: restauranttools.ai 7shifts pricing tracker (verified July 2, 2026), checkthat.ai 7shifts pricing breakdown (March 2026), stackscored.com scheduling pricing comparison (April 2026), shifton.com Homebase vs 7shifts, ontheclock.com 7shifts review (Feb 2026). Note: 7shifts rebranded plans in 2026 and trackers disagree on exact tier contents — confirm on their live page before quoting numbers in sales material.

The doc's headline claims check out: tip management ($49.99) and log book ($14.99) really are paid add-ons at 7shifts; restOS includes both in a $25 tier. **A venue wanting 7shifts Essentials + tips + log book pays ~$105–125/mo for what restOS Pro does at $25.**

Their pain points vs. your features

Documented competitor pain point	restOS answer	Assessment
Unreliable notifications — the #1 complaint across 7shifts ("shift pool notifications don't always work"), Deputy ("staff turning up to unconfirmed shifts… hundreds of hours lost"), Homebase ("do not receive notifications")	Push → SMS fallback router with delivery logging, quiet hours, per-user prefs	Directly targets the right wound. This is the strongest marketing angle you have.
Nickel-and-diming — tips, log book, tasks are add-ons; repeated price increases; per-user pricing exploding with headcount	Flat $25/$99, everything included, no per-user fees	Real and provable. Put the math on one slide.
Disconnected systems — Square's pieces don't integrate; 7shifts can't do ordering; ordering companies can't do scheduling	One database; capacity and call-out features are only possible *because* it's one system	Structurally true, not marketing. 7shifts would have to build or buy an ordering company to copy smart capacity.
Labor forecasting locked at high tiers — 7shifts' POS-driven labor tools need Pro/Premium ($80–150) + sometimes POS API fees ($50/mo for Toast)	Labor-vs-revenue at $99 *including the ordering side*	True — but see 4.1: competitors' versions read **real POS sales**; yours currently reads online orders only. Their expensive feature is more accurate than your cheap one until you fix the revenue denominator.
Clunky/dated UX, steep learning curve (Deputy, 7shifts mobile complaints, separate 7punches app for clock-in)	Employee-first, 3-tap-max, dark M3, one-screen dashboard	Plausible advantage, unproven until real employees use it daily.
Auto-scheduling ignores availability (documented Deputy limitation)	Guardrails warn on overtime/minor-hours/consecutive days before publish	Different approach (advisory vs. auto) — arguably better for a 15-person restaurant where the manager knows everyone.

Where the moat claims are overconfident

- "No competitor does this" (smart capacity, call-out engine): true today for SMB tools, but 7shifts integrates with 65+ POS systems and could approximate capacity-throttling via a POS partner. The durable moat isn't the feature — it's that **you own both sides of the data**, so your version is real-time and free. Say that instead.
- The free tier competes with 7shifts' free Comp tier and Homebase's free plan — both are legitimately good. Your free tier wins on restaurant-specificity (tips, QR clock-in) but the wedge is really the **pilot relationship and the upgrade path**, not the free tier itself.

---

6. What It Needs — Prioritized

Before the first paying pilot (blocking): 1. Manual daily revenue entry so labor-% is real (half a day, fixes 4.1) 2. `place_order` RPC: atomic order+items, server-recomputed totals, rate limiting (fixes 4.2, 4.3) 3. Wire `autoAdjust()` to a server-side trigger; count kitchen roles only (fixes 4.4) 4. Test public checkout end-to-end against a real Supabase project, not demo mode

Before scale (important): 5. The apex_v2 carryovers: staff `user_id` migration, tip transaction, QR rotation, offline-queue hardening 6. Online payment (Stripe) as an option alongside manual 7. Square POS integration (the deferred Phase 4 — pull it forward once the pilot proves value; it's what makes the OS number automatic instead of entered) 8. CI running analyze + tests on every PR

Later (the plan already says this — correctly): 9. Prep-time ML from captured snapshots 10. Multi-location tier 11. Vertical two — only when vertical one pays

---

7. Bottom Line

The design is better than most funded startups produce: one database, RLS everywhere, phased build order, honest docs, and OS features that are genuinely impossible for the scheduling-only and ordering-only competitors to replicate without an acquisition. The pricing ($25/$99 flat against a real ~$105–125/mo 7shifts equivalent) is aggressive and defensible.

It would work. The risks are not architectural — they're the four pre-pilot defects above (one broken checkout path, one untrusted price, one unwired auto-pause, one missing revenue denominator) plus the normal risk that a one-person company is the whole support department. Fix the four, run the Jigsy's pilot, measure the labor-cost delta the docs promise to measure, and the pitch writes itself with real numbers.

Audit performed against a local clone of the repo. Code findings reference commit `ffd8f5e`. Competitor pricing from the web sources listed in Section 5, retrieved July 27, 2026 — prices change; re-verify before quoting in sales material.